

Actividad Integradora

Consultas multitas y avanzadas, Vistas y Procedimientos Almacenados

Unidad: _____ RA3 _____

Estudiante: Eduardo Ventura Cepurniece

Centro/Institución: _____ INFOTEP _____

Matrícula/ID: 40225534227

Fecha: 23 de Junio de 2026 | **Ciudad:** ____ Santo Domingo _____

Indice

Introducción	4
Parte I. EVALUACIÓN TEÓRICA.....	5
1. Consultas Multitablas	5
a) ¿Cuál es la función principal de un INNER JOIN?	5
b) Diferencia entre INNER JOIN y LEFT JOIN.....	5
c) ¿En qué situaciones usaría un FULL JOIN?	5
d) ¿Ventajas de los alias de tablas?.....	5
2. Consultas Avanzadas	5
a) ¿Qué es una subconsulta?	5
b) Dos ventajas de consultas anidadas	6
c) Diferencia entre UNION y UNION ALL.....	6
d) ¿Función de GROUP BY?	6
3. Vistas	6
a) ¿Qué es una vista?.....	6
b) Tres ventajas de utilizar vistas	6
c) ¿Por qué es una tabla virtual?	6
d) Diferencia entre vista y tabla física.....	7
4. Procedimientos Almacenados	7
a) ¿Qué es un procedimiento almacenado?	7
b) Ventajas frente a consultas individuales	7
c) Diferencia entre parámetros de entrada y salida	7
d) Dos casos de uso	7
Parte II. Desarrollo Práctico	8
Sección A. Consultas Multitablas	8
Ejercicio 1.....	8
Ejercicio 2.....	9
Ejercicio 3.....	9
Sección B. Consultas Avanzadas	10

Ejercicio 4.....	10
Ejercicio 5.....	10
Ejercicio 6.....	11
Ejercicio 7.....	11
Sección C. Creación de Vistas	12
Ejercicio 8.....	12
Ejercicio 9.....	12
Ejercicio 10.....	13
Sección D. Procedimientos Almacenados	13
Ejercicio 11.....	13
Ejercicio 12.....	14
Ejercicio 13.....	14
Sección E. Caso Integrador	15
Cliente con mayor volumen de compras.	15
Producto más vendido.....	15
Oficina con mayor cantidad de ventas.	16
Clientes que superan el promedio general de compras.	16
Vista para generar un reporte ejecutivo.	17
Procedimiento almacenado para consultar rápidamente los clientes destacados. ...	17
Bibliografía	18
Conclusión.....	19

Introducción

En el entorno actual, las bases de datos juegan un papel fundamental en la gestión y análisis de la información empresarial. Las organizaciones dependen cada vez más de sistemas que les permitan transformar datos en conocimiento útil para la toma de decisiones. En este contexto, el manejo adecuado de consultas multitaslas, consultas avanzadas, vistas y procedimientos almacenados se convierte en una competencia esencial para cualquier analista de bases de datos.

La presente actividad integradora tiene como propósito aplicar de manera práctica y teórica los conocimientos adquiridos durante esta unidad, simulando situaciones reales dentro de una empresa de distribución. A través del desarrollo de consultas, creación de vistas y automatización mediante procedimientos almacenados, se busca construir un sistema de inteligencia comercial capaz de generar información relevante, optimizar procesos y apoyar el análisis estratégico del negocio.

Parte I. EVALUACIÓN TEÓRICA

1. Consultas Multitablas

a) ¿Cuál es la función principal de un INNER JOIN?

Un INNER JOIN se utiliza para combinar registros de dos o más tablas mostrando solo los datos que tienen coincidencia en ambas tablas según una condición.

b) Diferencia entre INNER JOIN y LEFT JOIN

- INNER JOIN: Devuelve solo los registros que coinciden en ambas tablas.
- LEFT JOIN: Devuelve todos los registros de la tabla izquierda, aunque no tengan coincidencia en la tabla derecha (en ese caso, muestra valores nulos).

c) ¿En qué situaciones usaría un FULL JOIN?

Cuando necesitas ver todos los registros de ambas tablas, tanto los que coinciden como los que no. Es útil para análisis completos donde no quieres perder información.

d) ¿Ventajas de los alias de tablas?

- Hacen las consultas más cortas y legibles.
- Facilitan trabajar con varias tablas a la vez.
- Evitan confusión cuando hay nombres de columnas repetidos.

2. Consultas Avanzadas

a) ¿Qué es una subconsulta?

Es una consulta dentro de otra consulta que se usa para obtener datos que sirven como base para la consulta principal.

b) Dos ventajas de consultas anidadas

- Permiten resolver problemas complejos de forma estructurada.
- Ayudan a evitar múltiples consultas separadas, optimizando el trabajo.

c) Diferencia entre UNION y UNION ALL

- UNION: Combina resultados eliminando duplicados.
- UNION ALL: Combina resultados incluyendo todos los registros, incluso los duplicados (es más rápido).

d) ¿Función de GROUP BY?

Sirve para agrupar filas que tienen valores iguales en ciertas columnas y aplicar funciones como SUM, COUNT, AVG, etc.

3. Vistas

a) ¿Qué es una vista?

Es una consulta guardada que se puede usar como si fuera una tabla.

b) Tres ventajas de utilizar vistas

- Simplifican consultas complejas.
- Mejoran la seguridad al restringir acceso a ciertos datos.
- Permiten reutilizar consultas sin escribirlas de nuevo.

c) ¿Por qué es una tabla virtual?

Porque no almacena datos físicamente, sino que muestra resultados basados en una consulta.

d) Diferencia entre vista y tabla física

- Vista: No almacena datos, solo los consulta.
- Tabla: Guarda los datos de forma permanente en la base de datos.

4. Procedimientos Almacenados

a) ¿Qué es un procedimiento almacenado?

Es un conjunto de instrucciones SQL guardadas en la base de datos que se pueden ejecutar cuando se necesiten.

b) Ventajas frente a consultas individuales

- Mejor rendimiento (se ejecutan más rápido).
- Reutilización del código.
- Mayor seguridad al controlar accesos.

c) Diferencia entre parámetros de entrada y salida

- Entrada: Se utilizan para enviar datos al procedimiento.
- Salida: Se utilizan para devolver resultados del procedimiento.

d) Dos casos de uso

- Procesar transacciones complejas (por ejemplo, registrar una compra).
- Generar reportes automáticos en la base de datos.

Parte II. Desarrollo Práctico

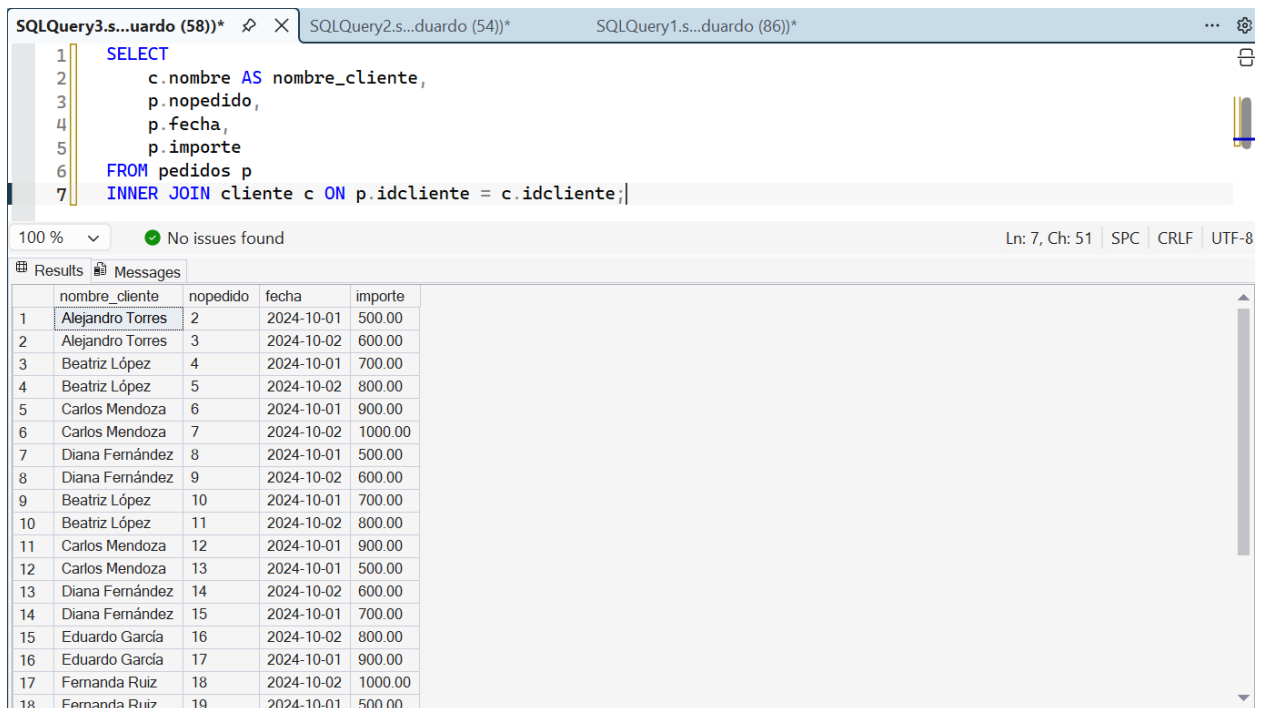
Sección A. Consultas Multitablas

Ejercicio 1

Construya una consulta que muestre:

- Nombre del cliente
- Número de pedido
- Fecha del pedido
- Importe del pedido

Utilice INNER JOIN.



The screenshot shows a SQL IDE with three tabs: 'SQLQuery3.s...duardo (58))*', 'SQLQuery2.s...duardo (54))*', and 'SQLQuery1.s...duardo (86))*'. The active tab 'SQLQuery3.s...duardo (58))*' contains the following SQL query:

```
1 SELECT
2     c.nombre AS nombre_cliente,
3     p.nopedido,
4     p.fecha,
5     p.importe
6 FROM pedidos p
7 INNER JOIN cliente c ON p.idcliente = c.idcliente;
```

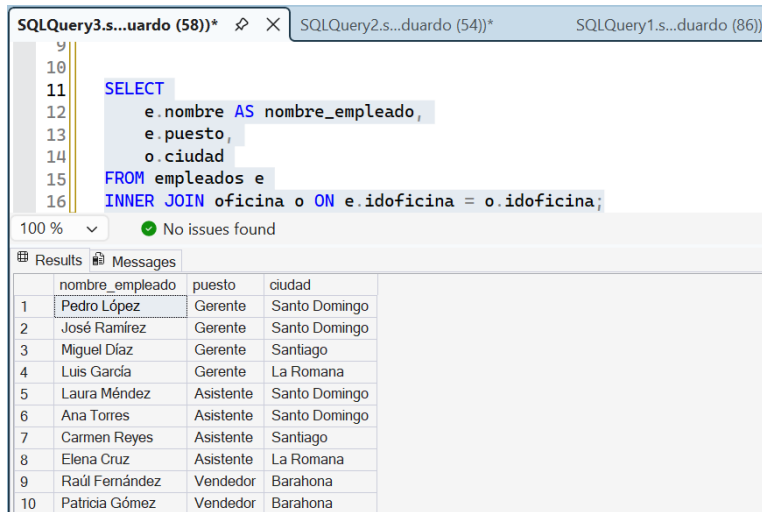
Below the query editor, the 'Results' tab is selected, displaying a table with 18 rows and 5 columns: 'nombre_cliente', 'nopedido', 'fecha', and 'importe'. The data is as follows:

	nombre_cliente	nopedido	fecha	importe
1	Alejandro Torres	2	2024-10-01	500.00
2	Alejandro Torres	3	2024-10-02	600.00
3	Beatriz López	4	2024-10-01	700.00
4	Beatriz López	5	2024-10-02	800.00
5	Carlos Mendoza	6	2024-10-01	900.00
6	Carlos Mendoza	7	2024-10-02	1000.00
7	Diana Fernández	8	2024-10-01	500.00
8	Diana Fernández	9	2024-10-02	600.00
9	Beatriz López	10	2024-10-01	700.00
10	Beatriz López	11	2024-10-02	800.00
11	Carlos Mendoza	12	2024-10-01	900.00
12	Carlos Mendoza	13	2024-10-01	500.00
13	Diana Fernández	14	2024-10-02	600.00
14	Diana Fernández	15	2024-10-01	700.00
15	Eduardo García	16	2024-10-02	800.00
16	Eduardo García	17	2024-10-01	900.00
17	Fernanda Ruiz	18	2024-10-02	1000.00
18	Fernanda Ruiz	19	2024-10-01	500.00

Ejercicio 2

Construya una consulta que muestre:

- Nombre del empleado
- Cargo
- Ciudad de la oficina donde trabaja
- Utilice las tablas correspondientes y los JOIN necesarios.



The screenshot shows a SQL query editor with a query window titled 'SQLQuery3.s...duardo (58))' and a results window titled 'Results'. The query is as follows:

```
SELECT
  e.nombre AS nombre_empleado,
  e.puesto,
  o.ciudad
FROM empleados e
INNER JOIN oficina o ON e.idoficina = o.idoficina;
```

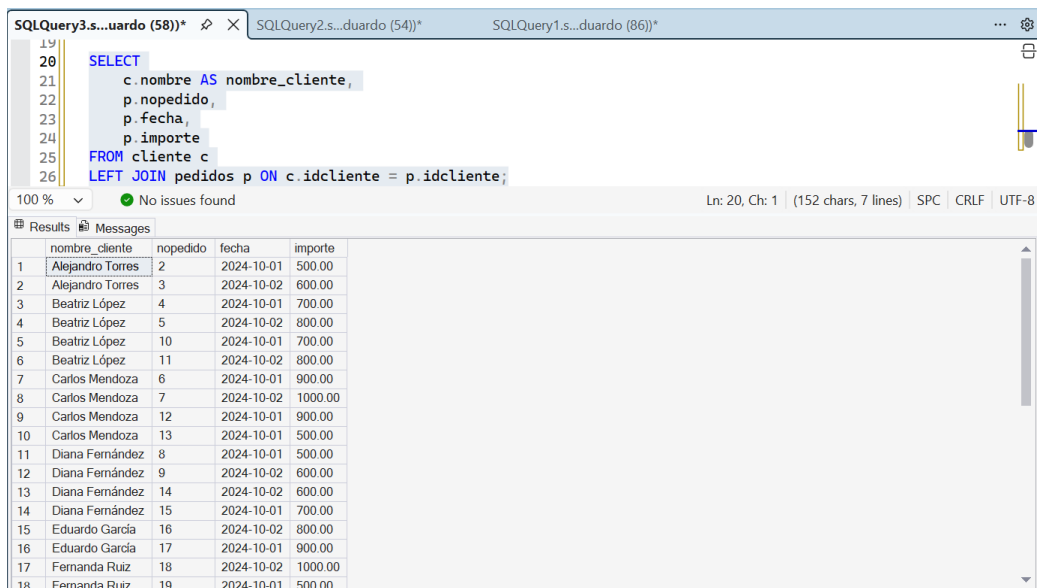
The results window shows the following data:

	nombre_empleado	puesto	ciudad
1	Pedro López	Gerente	Santo Domingo
2	José Ramírez	Gerente	Santo Domingo
3	Miguel Díaz	Gerente	Santiago
4	Luis García	Gerente	La Romana
5	Laura Méndez	Asistente	Santo Domingo
6	Ana Torres	Asistente	Santo Domingo
7	Carmen Reyes	Asistente	Santiago
8	Elena Cruz	Asistente	La Romana
9	Raúl Fernández	Vendedor	Barahona
10	Patricia Gómez	Vendedor	Barahona

Ejercicio 3

Muestre todos los clientes aunque no tengan pedidos registrados.

Utilice LEFT JOIN.



The screenshot shows a SQL query editor with a query window titled 'SQLQuery3.s...duardo (58))' and a results window titled 'Results'. The query is as follows:

```
SELECT
  c.nombre AS nombre_cliente,
  p.nopedido,
  p.fecha,
  p.importe
FROM cliente c
LEFT JOIN pedidos p ON c.idcliente = p.idcliente;
```

The results window shows the following data:

	nombre_cliente	nopedido	fecha	importe
1	Alejandro Torres	2	2024-10-01	500.00
2	Alejandro Torres	3	2024-10-02	600.00
3	Beatriz López	4	2024-10-01	700.00
4	Beatriz López	5	2024-10-02	800.00
5	Beatriz López	10	2024-10-01	700.00
6	Beatriz López	11	2024-10-02	800.00
7	Carlos Mendoza	6	2024-10-01	900.00
8	Carlos Mendoza	7	2024-10-02	1000.00
9	Carlos Mendoza	12	2024-10-01	900.00
10	Carlos Mendoza	13	2024-10-01	500.00
11	Diana Fernández	8	2024-10-01	500.00
12	Diana Fernández	9	2024-10-02	600.00
13	Diana Fernández	14	2024-10-02	600.00
14	Diana Fernández	15	2024-10-01	700.00
15	Eduardo García	16	2024-10-02	800.00
16	Eduardo García	17	2024-10-01	900.00
17	Fernanda Ruiz	18	2024-10-02	1000.00
18	Fernanda Ruiz	19	2024-10-01	500.00

Sección B. Consultas Avanzadas

Ejercicio 4

Obtenga el total comprado por cada cliente utilizando:

- SUM()
- GROUP BY

The screenshot shows a SQL Query Editor window with a query to calculate the total amount purchased by each client. The query uses the SUM() function and GROUP BY clause. The results are displayed in a table with two columns: nombre and total_comprado.

```
1 SELECT
2     c.nombre,
3     SUM(p.importe) AS total_comprado
4 FROM cliente c
5 INNER JOIN pedidos p ON c.idcliente = p.idcliente
6 GROUP BY c.nombre;
```

	nombre	total_comprado
1	Alejandro Torres	1100.00
2	Beatriz López	3000.00
3	Carlos Mendoza	3300.00
4	Diana Fernández	2400.00
5	Eduardo García	1700.00
6	Fernanda Ruiz	1500.00
7	Gabriel Martínez	1300.00
8	Helena Castro	1700.00
9	Ignacio Pérez	1500.00
10	Julia Morales	1900.00

Ejercicio 5

Obtenga la cantidad de pedidos registrados por cada cliente utilizando:

- COUNT()
- GROUP BY

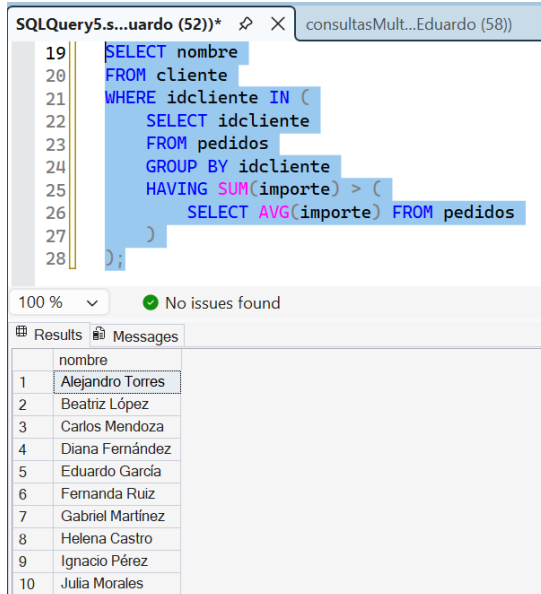
The screenshot shows a SQL Query Editor window with a query to count the number of orders registered by each client. The query uses the COUNT() function and GROUP BY clause. The results are displayed in a table with two columns: nombre and cantidad_pedidos.

```
10 SELECT
11     c.nombre,
12     COUNT(p.nopedido) AS cantidad_pedidos
13 FROM cliente c
14 INNER JOIN pedidos p ON c.idcliente = p.idcliente
15 GROUP BY c.nombre;
```

	nombre	cantidad_pedidos
1	Alejandro Torres	2
2	Beatriz López	4
3	Carlos Mendoza	4
4	Diana Fernández	4
5	Eduardo García	2
6	Fernanda Ruiz	2
7	Gabriel Martínez	2
8	Helena Castro	2
9	Ignacio Pérez	2
10	Julia Morales	2

Ejercicio 6

Utilice una subconsulta para identificar los clientes cuyo volumen de compras sea superior al promedio general de ventas.



```
SQLQuery5.s...uardo (52))* X consultasMult...Eduardo (58))
19 SELECT nombre
20 FROM cliente
21 WHERE idcliente IN (
22     SELECT idcliente
23     FROM pedidos
24     GROUP BY idcliente
25     HAVING SUM(importe) > (
26         SELECT AVG(importe) FROM pedidos
27     )
28 );
```

100 % No issues found

	nombre
1	Alejandro Torres
2	Beatriz López
3	Carlos Mendoza
4	Diana Fernández
5	Eduardo García
6	Fernanda Ruiz
7	Gabriel Martínez
8	Helena Castro
9	Ignacio Pérez
10	Julia Morales

Ejercicio 7

Realice una consulta utilizando UNION para combinar:

- Clientes
- Suplidores

Mostrando únicamente nombre y ciudad.



```
SQLQuery5.s...uardo (52))* X consultasMult...Eduardo (58)) SQLQuery2.s...uardo (54))* SQLQuery1.s...uardo (86))*
31
32 -- CLIENTES
33 SELECT
34     c.nombre,
35     o.ciudad
36 FROM cliente c
37 INNER JOIN oficina o ON c.idoficina = o.idoficina
38
39 UNION
40
41 -- SUPLIDORES
42 SELECT
43     s.nombre,
44     LTRIM(SUBSTRING(s.direccion, CHARINDEX(' ', s.direccion) + 1, LEN(s.direccion))) AS ciudad
45 FROM suplidor s;
```

100 % No issues found Ln: 33, Ch: 1 (267 chars, 13 lines) SPC CRLF UTF-8

	nombre	ciudad
1	Alejandro Montaña	Punta Cana
2	Alejandro Torres	Santo Domingo
3	Beatriz López	Santo Domingo
4	Carlos Mendoza	Santo Domingo
5	Diana Fernández	Santo Domingo
6	Dioris Rojas	La Romana
7	Eduardo García	Santiago
8	Fernanda Ruiz	Santiago
9	Gabriel Martínez	Santiago

Sección C. Creación de Vistas

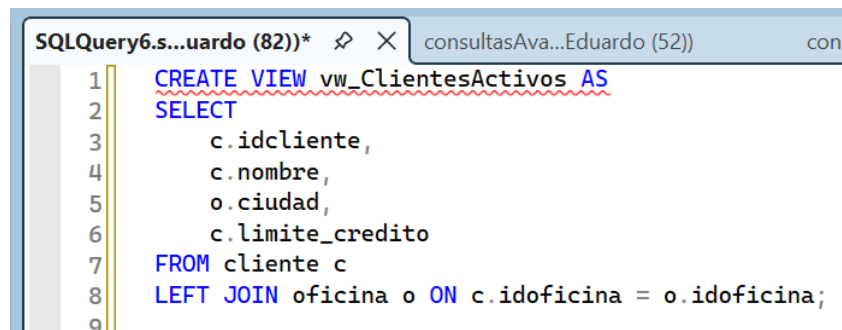
Ejercicio 8

Crear una vista denominada:

- vw_ClientesActivos

Debe mostrar:

- Código del cliente
- Nombre
- Ciudad
- Límite de crédito



```
SQLQuery6.s...uardo (82))* consultasAva...Eduardo (52)) con:
1 CREATE VIEW vw_ClientesActivos AS
2 SELECT
3     c.idcliente,
4     c.nombre,
5     o.ciudad,
6     c.limite_credito
7 FROM cliente c
8 LEFT JOIN oficina o ON c.idoficina = o.idoficina;
```

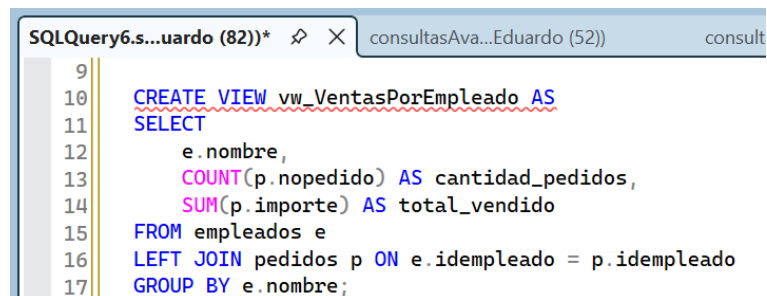
Ejercicio 9

Crear una vista denominada:

- vw_VentasPorEmpleado

Debe mostrar:

- Nombre del empleado
- Cantidad de pedidos gestionados
- Total vendido



```
SQLQuery6.s...uardo (82))* consultasAva...Eduardo (52)) consult
9
10 CREATE VIEW vw_VentasPorEmpleado AS
11 SELECT
12     e.nombre,
13     COUNT(p.nopedido) AS cantidad_pedidos,
14     SUM(p.importe) AS total_vendido
15 FROM empleados e
16 LEFT JOIN pedidos p ON e.idempleado = p.idempleado
17 GROUP BY e.nombre;
```

Ejercicio 10

Ejecutar consultas sobre las vistas creadas y mostrar evidencias de los resultados.

```
18
19 SELECT * FROM vw_ClientesActivos;
20
21 SELECT * FROM vw_VentasPorEmpleado;
```

100 % 2 0

Results Messages

	idcliente	nombre	ciudad	limite_credito
1	1	Alejandro Torres	Santo Domingo	10000.00
2	2	Beatriz López	Santo Domingo	15000.00
3	3	Carlos Mendoza	Santo Domingo	20000.00
4	4	Diana Fernández	Santo Domingo	25000.00
5	5	Eduardo García	Santiago	12000.00
6	6	Fernanda Ruiz	Santiago	18000.00
7	7	Gabriel Martínez	Santiago	22000.00
8	8	Helena Castro	Santiago	27000.00

	nombre	cantidad_pedidos	total_vendido
1	Ana Torres	3	2900.00
2	Carmen Reyes	2	1500.00
3	Elena Cruz	2	1100.00
4	José Ramírez	3	2000.00
5	Laura Méndez	3	2600.00
6	Luis García	3	2300.00

Sección D. Procedimientos Almacenados

Ejercicio 11

Crear un procedimiento almacenado llamado:

- sp_ListarClientes
- Que muestre todos los clientes.

SQLQuery11....uardo (53)) SQLQuery10.s...Eduardo (88))

```
1 USE [almacen]
2 GO
3
4 DECLARE @return_value int
5
6 EXEC @return_value = [dbo].[sp_ListarClientes]
7
8 SELECT 'Return Value' = @return_value
9
10 GO
11
```

100 % 1 0

Results Messages

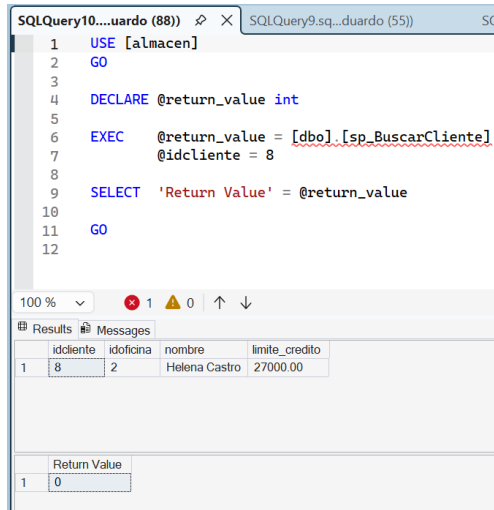
	idcliente	idoficina	nombre	limite_credito
1	1	1	Alejandro Torres	10000.00
2	2	1	Beatriz López	15000.00
3	3	1	Carlos Mendoza	20000.00
4	4	1	Diana Fernández	25000.00
5	5	2	Eduardo García	12000.00
6	6	2	Fernanda Ruiz	18000.00
7	7	2	Gabriel Martínez	22000.00
8	8	2	Helena Castro	27000.00
9	9	3	Innacio Pérez	13000.00

	Return Value
1	0

Ejercicio 12

Crear un procedimiento almacenado llamado:

- sp_BuscarCliente
- Debe recibir como parámetro el código del cliente.



The screenshot shows a SQL query window with the following code:

```
1 USE [almacen]
2 GO
3
4 DECLARE @return_value int
5
6 EXEC @return_value = [dbo].[sp_BuscarCliente]
7     @idcliente = 8
8
9 SELECT 'Return Value' = @return_value
10
11 GO
12
```

Below the code, the 'Results' tab displays the output of the stored procedure call:

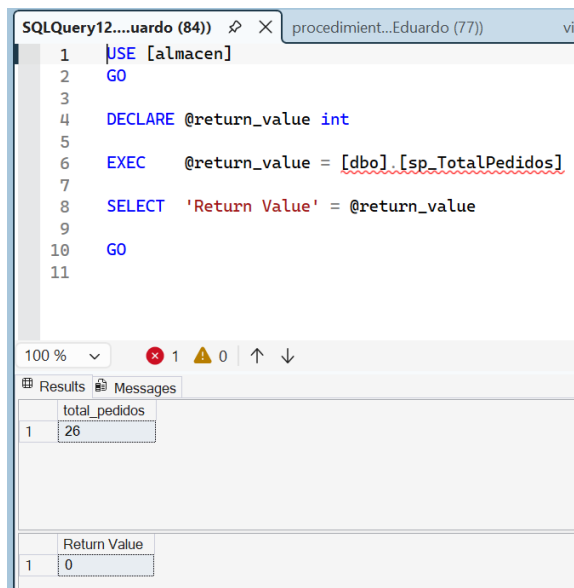
idcliente	idoficina	nombre	limite_credito
8	2	Helena Castro	27000.00

At the bottom, the 'Return Value' is shown as 0.

Ejercicio 13

Crear un procedimiento almacenado llamado:

- sp_TotalPedidos
- Debe devolver la cantidad total de pedidos registrados.



The screenshot shows a SQL query window with the following code:

```
1 USE [almacen]
2 GO
3
4 DECLARE @return_value int
5
6 EXEC @return_value = [dbo].[sp_TotalPedidos]
7
8 SELECT 'Return Value' = @return_value
9
10 GO
11
```

Below the code, the 'Results' tab displays the output of the stored procedure call:

total_pedidos
26

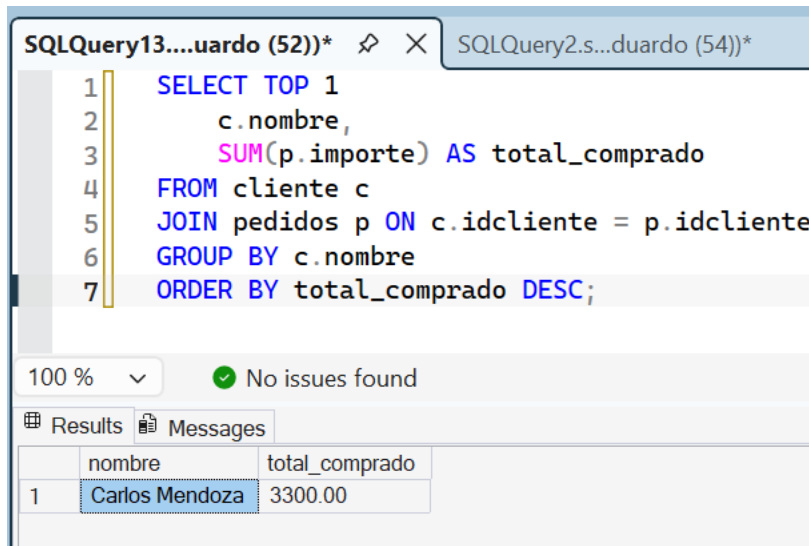
At the bottom, the 'Return Value' is shown as 0.

Sección E. Caso Integrador

La gerencia desea identificar oportunidades de negocio.

Desarrolle una solución que permita obtener:

Cliente con mayor volumen de compras.



The screenshot shows a SQL query window with the following text:

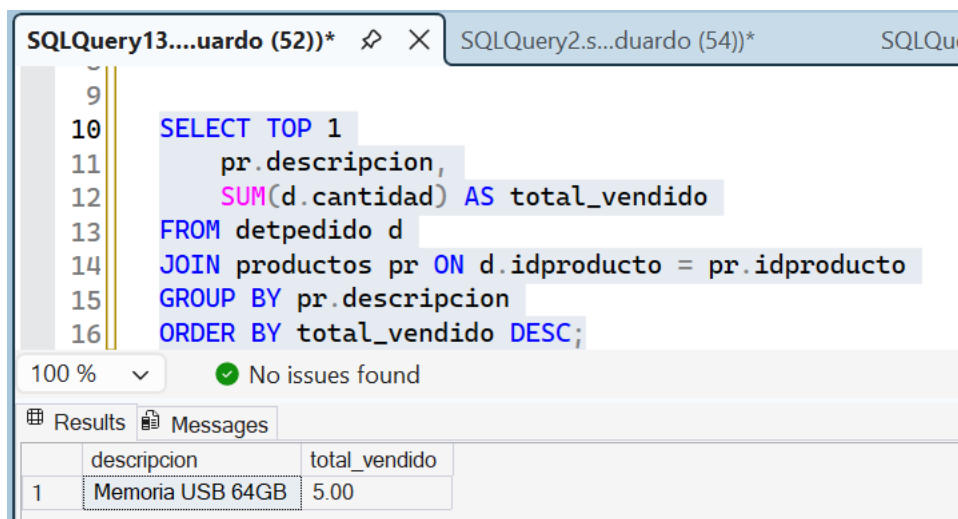
```
1 SELECT TOP 1
2     c.nombre,
3     SUM(p.importe) AS total_comprado
4 FROM cliente c
5 JOIN pedidos p ON c.idcliente = p.idcliente
6 GROUP BY c.nombre
7 ORDER BY total_comprado DESC;
```

Below the query, the 'Results' tab is active, displaying a table with two columns: 'nombre' and 'total_comprado'. The first row shows 'Carlos Mendoza' with a total of 3300.00.

	nombre	total_comprado
1	Carlos Mendoza	3300.00

Permite identificar el cliente más valioso para la empresa, lo que ayuda a diseñar estrategias de fidelización.

Producto más vendido.



The screenshot shows a SQL query window with the following text:

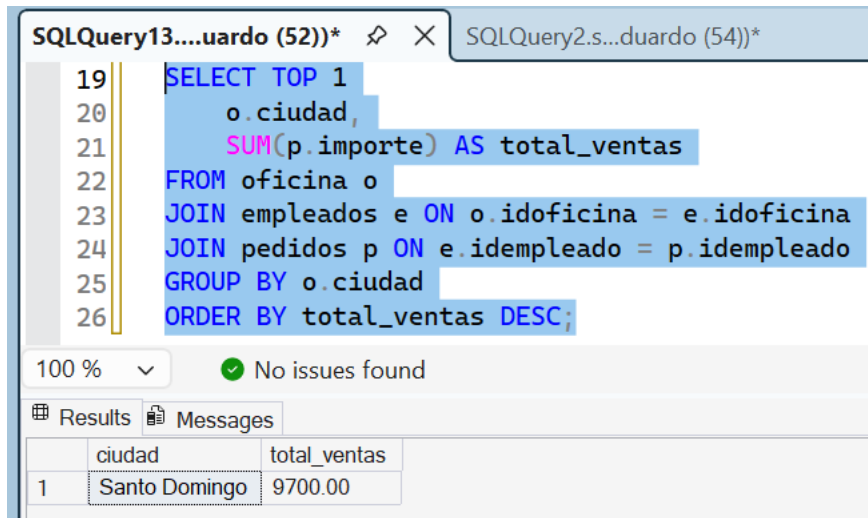
```
9
10 SELECT TOP 1
11     pr.descripcion,
12     SUM(d.cantidad) AS total_vendido
13 FROM detpedido d
14 JOIN productos pr ON d.idproducto = pr.idproducto
15 GROUP BY pr.descripcion
16 ORDER BY total_vendido DESC;
```

Below the query, the 'Results' tab is active, displaying a table with two columns: 'descripcion' and 'total_vendido'. The first row shows 'Memoria USB 64GB' with a total of 5.00.

	descripcion	total_vendido
1	Memoria USB 64GB	5.00

Muestra el producto con mayor demanda, útil para decisiones de inventario y compras.

Oficina con mayor cantidad de ventas.



The screenshot shows a SQL query window with the following text:

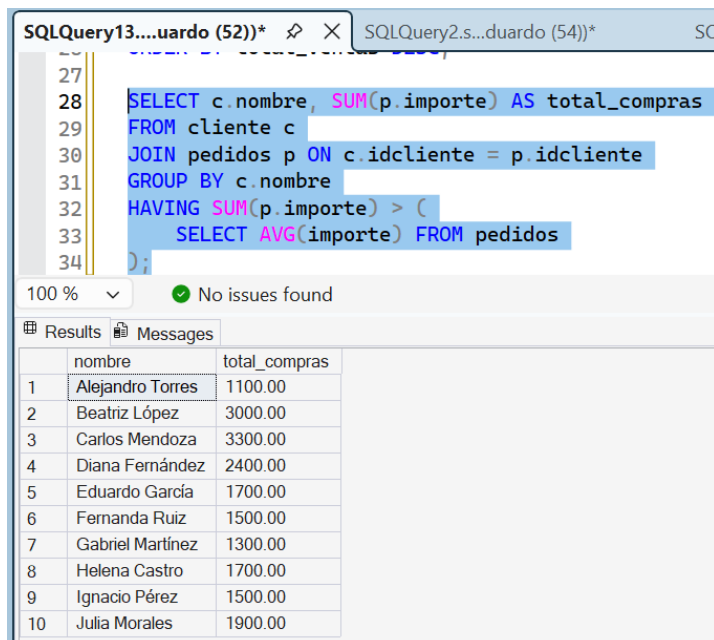
```
19 SELECT TOP 1
20     o.ciudad,
21     SUM(p.importe) AS total_ventas
22 FROM oficina o
23 JOIN empleados e ON o.idoficina = e.idoficina
24 JOIN pedidos p ON e.idempleado = p.idempleado
25 GROUP BY o.ciudad
26 ORDER BY total_ventas DESC;
```

Below the query, the status bar indicates "100 % No issues found". The "Results" tab is active, showing a table with two columns: "ciudad" and "total_ventas".

	ciudad	total_ventas
1	Santo Domingo	9700.00

Identifica la oficina con mejor rendimiento, apoyando decisiones estratégicas regionales.

Clientes que superan el promedio general de compras.



The screenshot shows a SQL query window with the following text:

```
27
28 SELECT c.nombre, SUM(p.importe) AS total_compras
29 FROM cliente c
30 JOIN pedidos p ON c.idcliente = p.idcliente
31 GROUP BY c.nombre
32 HAVING SUM(p.importe) > (
33     SELECT AVG(importe) FROM pedidos
34 );
```

Below the query, the status bar indicates "100 % No issues found". The "Results" tab is active, showing a table with two columns: "nombre" and "total_compras".

	nombre	total_compras
1	Alejandro Torres	1100.00
2	Beatriz López	3000.00
3	Carlos Mendoza	3300.00
4	Diana Fernández	2400.00
5	Eduardo García	1700.00
6	Fernanda Ruiz	1500.00
7	Gabriel Martínez	1300.00
8	Helena Castro	1700.00
9	Ignacio Pérez	1500.00
10	Julia Morales	1900.00

Permite detectar clientes por encima del promedio, ideales para promociones especiales.

Vista para generar un reporte ejecutivo.

SQLQuery14....duardo (72) X SQLQuery13.s...duardo (52))*

```
1 SELECT TOP (1000) [cliente]
2 , [cantidad_pedidos]
3 , [total_comprado]
4 , [oficina]
5 FROM [almacen].[dbo].[vw_ReporteEjecutivo]
6
```

100 % 5 0 ↑ ↓

Results Messages

	cliente	cantidad_pedidos	total_comprado	oficina
1	Ignacio Pérez	2	1500.00	La Romana
2	Julia Morales	2	1900.00	La Romana
3	Eduardo García	2	1700.00	Santiago
4	Fernanda Ruiz	2	1500.00	Santiago
5	Gabriel Martínez	2	1300.00	Santiago
6	Helena Castro	2	1700.00	Santiago
7	Alejandro Torres	2	1100.00	Santo Domingo
8	Beatriz López	4	3000.00	Santo Domingo
9	Carlos Mendoza	4	3300.00	Santo Domingo
10	Diana Fernández	4	2400.00	Santo Domingo

Centraliza información clave para la gerencia en un solo reporte simplificado.

Procedimiento almacenado para consultar rápidamente los clientes destacados.

SQLQuery15....duardo (53) X SQLQuery13.s...duardo (52))* SQLQuery

```
1 USE [almacen]
2 GO
3
4 DECLARE @return_value int
5
6 EXEC @return_value = [dbo].[sp_ClientesDestacados]
7
8 SELECT 'Return Value' = @return_value
9
10 GO
11
```

100 % 1 0 ↑ ↓

Results Messages

	nombre	total_compras
1	Alejandro Torres	1100.00
2	Beatriz López	3000.00
3	Carlos Mendoza	3300.00
4	Diana Fernández	2400.00
5	Eduardo García	1700.00
6	Fernanda Ruiz	1500.00
7	Gabriel Martínez	1300.00
8	Helena Castro	1700.00
9	Ignacio Pérez	1500.00
Return Value		
1	0	

Permite consultar rápidamente los clientes más importantes sin rehacer la consulta.

Bibliografía

Date, C. J. (2019). An Introduction to Database Systems. Pearson.

Elmasri, R., & Navathe, S. (2017). Fundamentals of Database Systems. Pearson.

Microsoft. (2024). Documentación oficial de SQL Server. Recuperado de:
<https://learn.microsoft.com>

Coronel, C., & Morris, S. (2016). Database Systems: Design, Implementation, & Management. Cengage Learning.

Silberschatz, A., Korth, H., & Sudarshan, S. (2020). Database System Concepts. McGraw-Hill.

Conclusión

La realización de esta actividad permitió integrar diversos conceptos fundamentales de bases de datos en un escenario práctico orientado al análisis empresarial. El uso de consultas multitaslas facilitó la combinación de información proveniente de diferentes fuentes, mientras que las consultas avanzadas permitieron obtener resultados más precisos y significativos mediante funciones de agregación y subconsultas.

Asimismo, la creación de vistas contribuyó a simplificar el acceso a la información y a mejorar la organización de los datos, mientras que los procedimientos almacenados demostraron ser herramientas clave para la automatización de tareas y la optimización del rendimiento del sistema. En conjunto, estos elementos permiten desarrollar soluciones eficientes que aportan valor a la gestión empresarial.

En definitiva, esta actividad no solo fortaleció las habilidades técnicas, sino también la capacidad de análisis y resolución de problemas, competencias indispensables para el desempeño profesional en el área de bases de datos.